Software Architecture and Design Specification

Project: Task Manager Application (SE Mini-Project)

Version: 1.0

Authors: Vismay H S (PES1UG24AM325), Trisha G Shetty (PES1UG24AM308), Akash Kiran Vernekar (PES1UG24AM338), Basawaraj Panchal (PES1UG24AM350)

Date: 01-10-2026

Status: Draft — for team review before submission

Revision History

Version | Date | Author | Change Summary

1.0 | 01-10-2026 | Vismay H S (PES1UG24AM325) | Initial SAD: architecture, component diagram, and sequence diagrams for the Task Manager Application, based on SRS_TaskManagerApplication.docx v1.3.

Approvals

Role | Name | Signature/Date

Team Member | Vismay H S (PES1UG24AM325) |

Team Member | Trisha G Shetty (PES1UG24AM308) |

Team Member | Akash Kiran Vernekar (PES1UG24AM338) |

Team Member | Basawaraj Panchal (PES1UG24AM350) |

Course Instructor | |

1. Introduction

1.1 Purpose

This document specifies the software architecture and component-level design of the Task Manager Application, translating the requirements in SRS_TaskManagerApplication.docx v1.3 into an implementable system structure.

1.2 Scope

Covers user registration & authentication (TM-F-001–003), task CRUD (TM-F-010–012), task assignment & the Kanban board (TM-F-020–023), notifications & comments (TM-F-030–032), and search/filter/reporting (TM-F-040–042), together with the performance,

reliability, and security requirements (TM-NF-001–005, PRJ-SR-001–005). Excludes hosting/infrastructure management (per SRS §1.2).

1.3 Audience
The four-member project team, the course instructor/evaluators, and QA.

1.4 Definitions
UI, API, REST, JWT, CRUD, SPA, ORM, RBAC (as defined in the SRS §1.4). SRS — Software Requirements Specification. RTM — Requirements Traceability Matrix.

2. Document Overview

2.1 How to use this document
Sections 3 and 4 are the architectural deliverables: the layered architecture, the UML component diagram, two UML sequence diagrams, the API design for two components, and the threat model. Read alongside SRS_TaskManagerApplication.docx v1.3 for full requirement traceability.

2.2 Related Documents
SRS_TaskManagerApplication.docx v1.3 (this repository, Templates branch), this SAD, the companion Software Test Plan (STP).

3. Architecture

3.1 Goals & Constraints
Goals: API response time under 2 seconds for 95% of requests (TM-NF-001); 99% monthly uptime (TM-NF-002); support at least 100 concurrent users without noticeable degradation (TM-NF-005). Constraints: must be delivered as a React.js frontend with a Node.js/Express REST API backend (SRS §2.5); project scope tracked in Jira; four-member team on a semester timeline.

3.2 Stakeholders & Concerns
Admin: user account management, full system access.
Project/Team Lead: creating projects, assigning tasks, viewing reports.
Team Member: viewing assigned tasks, updating status, commenting.
Course Evaluators: correctness against the SRS, architecture quality.
Developers/QA: modularity, testability, clear interfaces between components.

3.3 Component (UML) Diagram

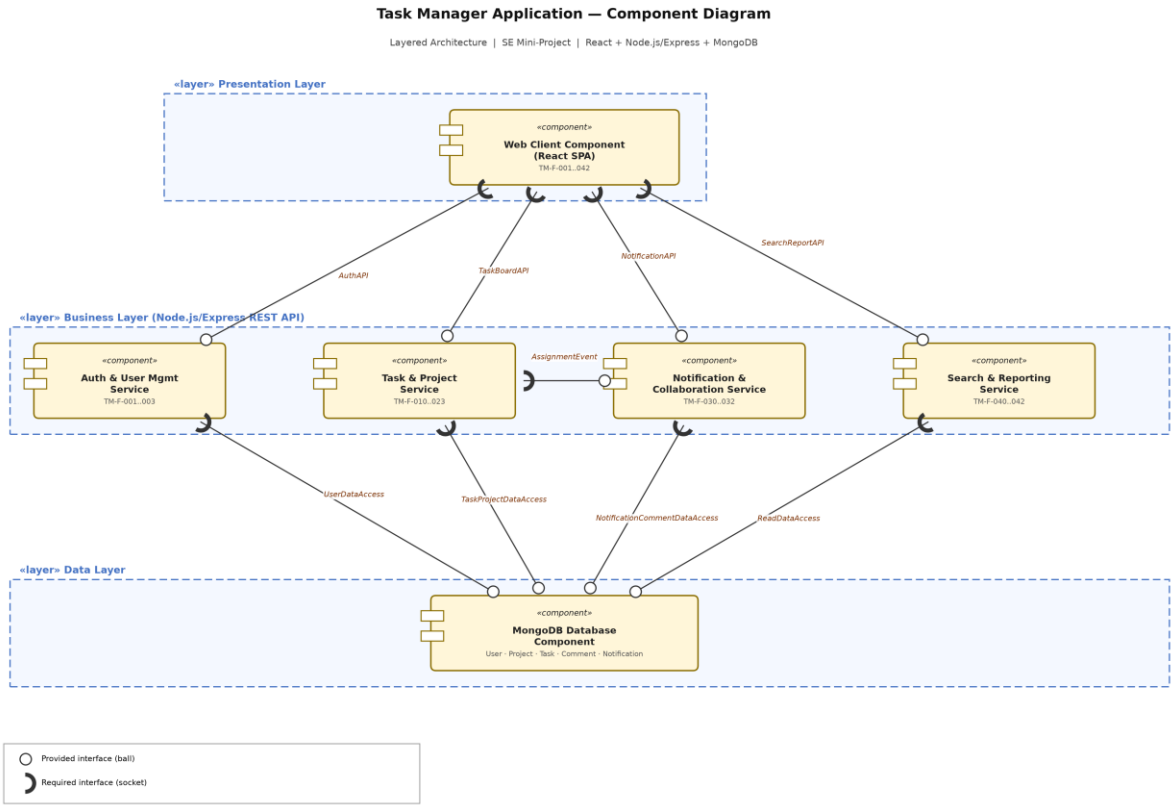


Figure 1 — UML Component Diagram: 6 components across Presentation / Business / Data layers, 9 ball-and-socket interfaces.

3.4 Component Descriptions

- Web Client Component (React SPA): the single responsive web UI — login/registration forms, dashboard, Kanban board, task forms, notifications (TM-F-001..042).
- Auth & User Mgmt Service: registration, login, JWT issuance, profile management (TM-F-001..003).
- Task & Project Service: task CRUD, project/board management, task assignment (TM-F-010..023).
- Notification & Collaboration Service: in-app notifications and task comments (TM-F-030..032).
- Search & Reporting Service: search, filters, dashboard summaries, CSV export (TM-F-040..042).
- MongoDB Database Component: single authoritative store for the User, Project, Task, Comment, and Notification collections (SRS §10).

3.5 Chosen Architecture Pattern and Rationale

Layered architecture (Presentation / Business / Data). Chosen because: (1) the SRS explicitly constrains delivery to a single React.js SPA talking to a single Node.js/Express

REST API backend and a single MongoDB store (§2.5) — a natural three-tier shape, not a distributed one; and (2) a four-person team on a semester timeline benefits from low operational overhead — module boundaries (Auth / Task / Notification / Search) still let each teammate own one service independently inside the same codebase, without the deployment and coordination cost of real microservices. Microservices was considered and rejected: the SRS's own assumptions (§9) state data volumes are "small-to-moderate (course project scale)", so the independent-scaling benefit microservices would offer isn't needed here. Client-Server alone was rejected because it under-specifies the internal module separation the component diagram needs to show.

3.6 Technology Stack & Data Stores

Frontend: React.js single-page application.
Backend: Node.js/Express REST API, organized into four internal service modules (Auth, Task & Project, Notification & Collaboration, Search & Reporting) communicating over HTTPS/JSON.
Data: MongoDB via Mongoose, holding the User, Project, Task, Comment, and Notification collections (SRS §10).
Security: JWT session tokens, bcrypt password hashing, HTTPS/TLS 1.2+ for all traffic.
Optional: Nodemailer for email due-date reminders (SRS §3.3).

3.7 Risks & Mitigations

Risk: MongoDB's flexible schema drifts as requirements evolve during the semester (the SRS explicitly assumes a schema-flexible DB, §9) → Mitigation: Mongoose schema validation plus versioned migration notes before each sprint.
Risk: four teammates working on shared modules (auth, task service, board UI) causes merge conflicts or integration breakage → Mitigation: the four Business-layer services (Auth, Task & Project, Notification, Search) give each teammate one independently-ownable module; Jira tracks who owns what.
Risk: API latency degrades past the TM-NF-001 2-second target as task/notification volume grows → Mitigation: MongoDB indexes on projectId/assigneeId/status, pagination on list endpoints.
Risk: JWT leakage or XSS on the React SPA → Mitigation: secure token storage, input sanitization (PRJ-SR-005), CSP headers.

3.8 Traceability to Requirements

TM-F-001..003 (auth & user mgmt) → Web Client + Auth & User Mgmt Service
TM-F-010..012 (task CRUD) → Web Client + Task & Project Service
TM-F-020..023 (assignment & board) → Web Client + Task & Project Service
TM-F-030..032 (notifications & comments) → Web Client + Notification & Collaboration Service
TM-F-040..042 (search/filter/reporting) → Web Client + Search & Reporting Service
TM-NF-001 (≤2s API response) → all four Business-layer services
TM-NF-002 (99% uptime) → Business layer + MongoDB Database
TM-NF-005 (100 concurrent users) → Business layer + MongoDB Database

PRJ-SR-001/002 (password hashing, JWT) → Auth & User Mgmt Service
PRJ-SR-003 (RBAC) → all four Business-layer services
PRJ-SR-004/005 (TLS, input sanitization) → Web Client + all Business-layer services

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → JWT-based authenticated sessions, passwords hashed with bcrypt (PRJ-SR-001). Tampering → no Presentation-layer component writes MongoDB directly; every write is validated inside a named Business-layer interface. Repudiation → createdAt/updatedAt timestamps and an authorId on every Task, Comment, and Notification record. Information Disclosure → HTTPS/TLS 1.2+ for all traffic (PRJ-SR-004); passwordHash is never returned in API responses. Denial of Service → rate limiting on the /auth/login and /auth/register endpoints to resist brute-force attempts. Elevation of Privilege → RBAC enforced server-side (PRJ-SR-003) so a Team Member cannot view or edit a project/task they are not a member of (403 on non-member requests).

4. Design

4.1 Design Overview

The system is designed as a single Presentation-layer client (the React SPA) talking only to four Business-layer services, which are the only components allowed to read or write the single MongoDB Data-layer store. This keeps each SRS feature area's logic in exactly one service and lets the client stay thin, matching the component diagram in Section 3.3.

4.2 UML Sequence Diagrams

At least 2 sequence diagrams covering 2 flows specific to this project:

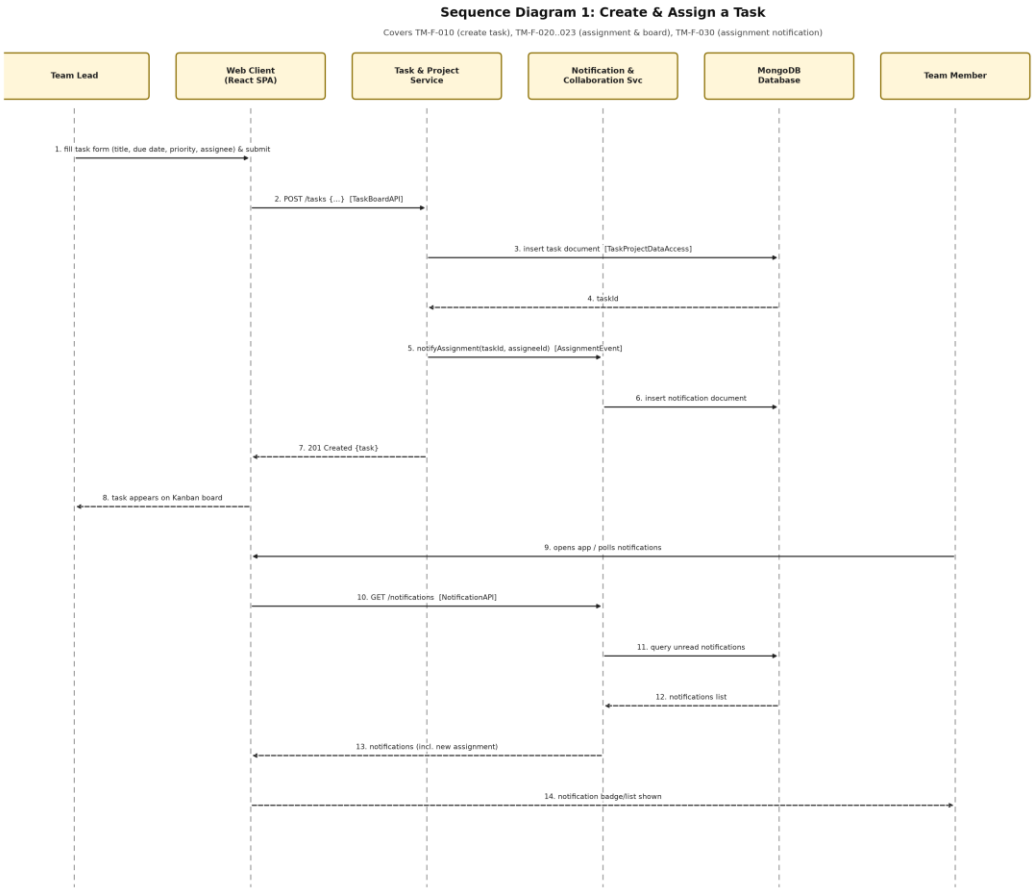


Figure 2 — Create & Assign a Task (TM-F-010, TM-F-020..023, TM-F-030).

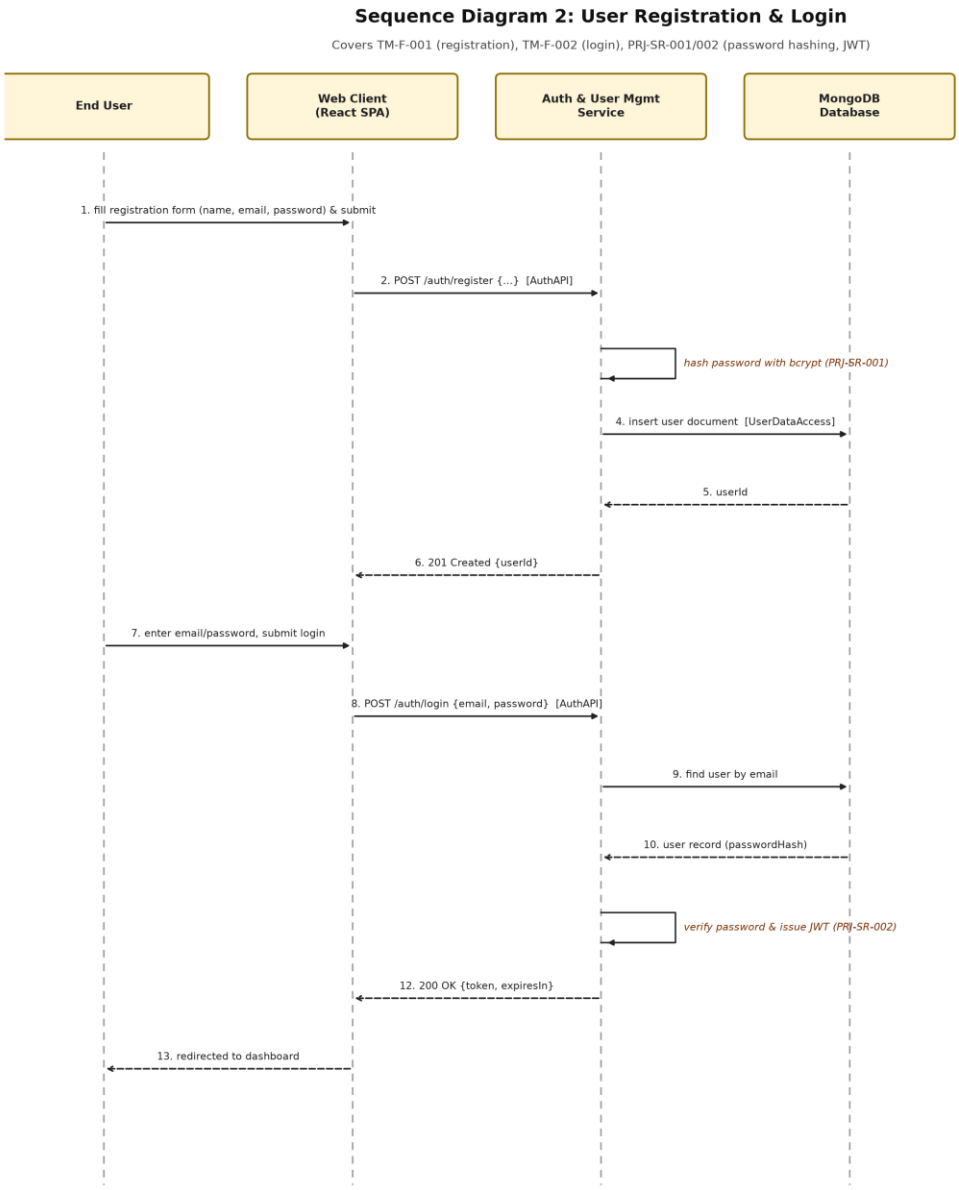


Figure 3 — User Registration & Login (TM-F-001, TM-F-002, PRJ-SR-001/002).

4.3 API Design

Interface definition for two Business-layer components:

Component: Auth & User Mgmt Service — interface AuthAPI
Endpoint: POST /auth/register
Request: {name, email, password}
Response: {userId, email}

Errors: 400 invalid email/weak password, 409 email already registered

Endpoint: POST /auth/login
Request: {email, password}
Response: {token, expiresIn}
Errors: 401 invalid credentials

Component: Task & Project Service — interface TaskBoardAPI
Endpoint: POST /tasks
Request: {title, description, dueDate, priority, projectId, assigneeIds[]}
Response: {taskId, status: "To Do"}
Errors: 400 missing required fields, 403 not a project member, 404 projectId not found

Endpoint: PATCH /tasks/{taskId}/status
Request: {status}
Response: {taskId, status}
Errors: 400 invalid status value, 403 not authorized, 404 task not found

4.4 Error Handling, Logging & Monitoring

All Business-layer APIs return a standardized error envelope: {errorCode, message, timestamp}. Passwords and JWTs are never written to application logs — only user IDs and emails for auth events. Monitoring dashboards track: API response latency against the TM-NF-001 2-second (p95) target, uptime against the TM-NF-002 99% target, and concurrent-user load against the TM-NF-005 100-user target.

4.5 UX Design

A responsive React UI (TM-NF-004) with a Kanban board (To Do / In Progress / Done), readable font sizes, keyboard navigation, and sufficient color contrast. Task creation/editing uses a simple modal form; notifications appear as an in-app badge/list rather than requiring a page reload.

4.6 Open Issues & Next Steps

Directly carried over from the SRS's own Future Scope (§11), not required for v1.0: file attachments on tasks, recurring tasks, a calendar view, email notifications in addition to in-app ones, dark mode, and a basic per-task activity/audit log.

5. Appendices

5.1 Glossary: UI, API, REST, JWT, CRUD, SPA, ORM, RBAC, SRS, RTM.
5.2 References: SRS_TaskManagerApplication.docx v1.3 (this repository), IEEE 42010, OWASP Top 10.
5.3 Tools: Python/matplotlib (diagrams), python-docx, Jira (defect/requirement tracking), draw.io UML notation conventions.